

Deploy: From Your Laptop to a Public URL

Vercel + Render + Neon — Three Free Services, One Public Link

CSCI 39548 — Practical Web Development

Where We Are Tonight

By now you have:

- ▶ A **React** + **Vite** frontend
- ▶ An **Express** + **Prisma** backend
- ▶ A **Postgres** database (Neon, from Prisma snapshot 04)

All of this works. All of it runs on **your laptop**.

Close the laptop, the app is dead. Tonight: how to give your app a real public URL.

What Does “Deploy” Actually Mean?

Deploying = moving your app from your laptop onto an always-on computer somewhere on the internet, so any browser in the world can reach it.

- ▶ It has a public URL (`https://your-app.vercel.app`)
- ▶ It's running 24/7 (your laptop can be closed)
- ▶ Anyone on any network can use it
- ▶ You don't have to be there

This is the difference between “I coded an app” and “I shipped an app.”

Why Not Just Keep Running `npm run dev`?

`npm run dev` runs your app on **your machine**, on **your network**, accessible only at `localhost`.

- ▶ `localhost` = your computer talking to itself. No one else can see it.
- ▶ Your IP address changes every time you switch networks.
- ▶ Your laptop sleeps. Your battery dies.

Cloud services handle the “always on, public, secure” part. You write your app; they handle the infrastructure.

The New Mental Model

Old (development):

Your laptop

|

v

localhost:5173

localhost:3000

dev.db / Neon

New (production):

Vercel

(Frontend, on a CDN)

|

v

Render

(Backend, in a container)

|

v

Neon

(Postgres, managed)

|

v

Available 24/7 at a public HTTPS

URL

Same code. Different hosts. Each piece moves from your laptop to a service.

What You Need Before You Start

1. Your app runs **locally** (frontend + backend + DB all working)
2. Your code is on **GitHub** (or you're about to push it)
3. Three free accounts (next slide)
4. Your `.env` is in `.gitignore` (never commit secrets)
5. You've made the 5 code changes that make the app “portable” (covered next)

*Note: If local doesn't work, **deploy won't fix it**. Test locally first.*

The “Deploy Can’t Fix Broken Local” Rule

If you push broken code to GitHub and click “Deploy,” the broken code runs in the cloud instead of on your laptop. That’s it.

Deploying does not:

- ▶ Fix bugs
- ▶ Install missing dependencies you forgot
- ▶ Connect to a database that doesn’t exist
- ▶ Make CORS issues go away

Rule: if `npm run dev` doesn’t work, stop and fix that first. Only deploy code that already works locally.

The Three Free Accounts

You need three GitHub-linked accounts. All three are free for our use:

Service	Sign up at	What it hosts
Neon	<code>neon.tech</code>	Your Postgres database
Render	<code>render.com</code>	Your Express backend
Vercel	<code>vercel.com</code>	Your React frontend

All three “Sign in with GitHub” — one click each. 5 minutes total.

The Big Picture

```
      Browser
      |
      v
Vercel (Frontend) --API--> Render (Backend) --Prisma--> Neon (DB)
```

Three services, one per layer:

- ▶ **Vercel** hosts the static React build
- ▶ **Render** hosts the Express server that talks to the DB
- ▶ **Neon** hosts the Postgres database

Each piece has a free tier sufficient for the final project.

Vercel: What It Does

Vercel = a hosting service optimized for static frontends.

- ▶ You give it a GitHub repo with a Vite/React app
- ▶ Vercel runs `npm run build` to produce static HTML/CSS/JS
- ▶ Uploads those files to a **CDN** (a global network of servers)
- ▶ Every browser gets your app from the CDN node closest to them

Vercel does **not** run your app's code at runtime — it just serves pre-built files. That's why it's cheap and fast.

Render: What It Does

Render = a hosting service for long-running web servers.

- ▶ You give it a GitHub repo with a Node.js app
- ▶ Render clones it, runs `npm install && npm run build`, then `npm run start`
- ▶ A public URL routes to that server

Think of it as: “a cloud computer running `npm run start` forever, with a public URL.”

Free tier sleeps after 15 min idle. First request after sleep $\approx$ 30s. Fine for class.

Neon: What It Does

Neon = Postgres-as-a-service.

- ▶ Sign up, click “Create project,” get a connection string
- ▶ Your Postgres database lives on Neon’s servers (AWS Virginia, Ohio, etc.)
- ▶ Your backend connects via the connection string

Free tier pauses compute after 5 min idle. First query after pause $\approx$ 1–2s. Fine for class.

Why Three Services Instead of One?

Separation of concerns. Each layer needs different infrastructure:

Layer	Needs	Best served by
Static files (HTML/CSS/JS)	Cheap, global, fast	CDN → Vercel
App server (Node.js)	Long-running, routable	Container → Render
Database (Postgres)	Persistent, transactional	Managed DB → Neon

Modern stacks split because each piece needs different things. That's the lesson.

You Can Swap Any of These

The **pattern** is platform-agnostic. The dashboard clicks are platform-specific.

Need	We use	Alternatives
Static frontend	Vercel	Netlify, Cloudflare Pages
Long-running backend	Render	Fly.io, Railway, Heroku
Managed Postgres	Neon	Supabase, Railway, AWS RDS

We pick these three because all are free, all integrate with GitHub in one click, and all have abundant tutorials.

The 5 Code Changes (Overview)

Before any deploy, 5 small edits make your app “portable”:

```
1. backend  package.json  -> add "build" script
2. backend  server.ts      -> read PORT from env var
3. backend  server.ts      -> read FRONTEND_ORIGIN from env var
4. frontend App.tsx        -> read VITE_API_URL from env var
5. both     .env.example    -> document which env vars are needed
```

Routes, schema, components, Zod, Prisma client — **all unchanged**. We just stop hardcoding values that depend on WHERE the app runs.

Why “5 Small Changes”?

The only thing that changes between “running on my laptop” and “running on Render” is **values that depend on the environment**:

- ▶ The port number to listen on
- ▶ The database URL to connect to
- ▶ The allowed origin for CORS
- ▶ The API URL the frontend should call

The big idea: code that runs anywhere reads its config from the environment. Code locked to one machine has those values hardcoded.

Edit 1: package.json Build Script

```
{
  "scripts": {
    "dev": "tsx watch src/server.ts",
    "build": "npx prisma generate && npx prisma migrate deploy",
    "start": "tsx src/server.ts"
  }
}
```

- ▶ Render runs `npm run build` once per deploy
- ▶ `prisma migrate deploy` keeps the Neon schema in sync with your code
- ▶ Schema change → push → build step applies migration automatically

Edit 2: server.ts — PORT from Env

Before:

```
const PORT = 3000;
```

After:

```
const PORT = Number(process.env.PORT ?? 3000);
```

- ▶ Render assigns the port at runtime (e.g. 10000)
- ▶ Hardcoding 3000 → healthcheck never gets a response → deploy fails
- ▶ Locally, PORT is undefined → ?? 3000 fallback kicks in

Note: Never set PORT manually in Render's UI. Render injects it automatically.

Edit 3: server.ts — CORS Origin from Env

Before:

```
app.use(cors());    // allows ALL origins
```

After:

```
app.use(  
  cors(  
    origin: process.env.FRONTEND_ORIGIN  
      ?? "http://localhost:5173",  
  })  
);
```

In production: allow only YOUR Vercel URL. Locally: fallback matches Vite dev server.

Why CORS Exists at All

CORS = Cross-Origin Resource Sharing. A browser security rule.

- ▶ When `your-frontend.vercel.app` (origin A) tries to fetch from `your-backend.onrender.com` (origin B), the browser asks the backend: “Are you OK with origin A calling you?”
- ▶ If the backend doesn't say yes, the browser **blocks the response**
- ▶ `cors({ origin: "https://..." })` is the backend saying yes to exactly that one origin

Without CORS configured, deployed frontends can never talk to deployed backends. This is the #1 deploy bug.

Edit 4: Frontend — API URL from Env

Before:

```
const API = "http://localhost:3000";
```

After:

```
const API = import.meta.env.VITE_API_URL  
  ?? "http://localhost:3000";
```

- ▶ Locally: VITE_API_URL unset → falls back to localhost:3000
- ▶ On Vercel: set VITE_API_URL → build bakes it into the bundle

Note: Vite strips any env var NOT starting with VITE_. Security feature.

Edit 5: .env.example Files

backend/.env.example:

```
DATABASE_URL=  
FRONTEND_ORIGIN=  
# Render assigns PORT automatically. Do NOT set it.
```

frontend/.env.example:

```
VITE_API_URL=http://localhost:3000
```

Documents which env vars the project needs. Real .env files stay in .gitignore — never committed.

In both backend/ and frontend/ (or one at repo root):

```
node_modules
.env
.env.local
dist
*.log
*.db
```

- ▶ Committing `.env` exposes your database password on GitHub
- ▶ Committing `node_modules` makes your repo huge and slow

Note: If you ever commit a real password by accident, rotate it immediately (regenerate in Neon). GitHub history is permanent.

Local Sanity Check (Why)

Before pushing to GitHub:

If your app doesn't work locally with the new env-driven shape, **deploy will not fix it.** You'll just spend 5 minutes watching Render build broken code.

Always sanity-check the 5 edits work locally first.

This is the cheapest moment to find bugs. Render builds take 3 minutes each; local restarts take 3 seconds.

Local Sanity Check (Backend)

Terminal A:

```
cd backend
cp .env.example .env
# edit .env: paste your Neon DATABASE_URL
# edit .env: FRONTEND_ORIGIN=http://localhost:5173
npm install
npm run dev
```

Expected:

```
Prisma demo listening on http://localhost:3000
```

If server crashes with `PrismaClientInitializationError`, your `DATABASE_URL` is wrong or empty.

Local Sanity Check (Frontend)

Terminal B (leave backend running):

```
cd frontend
cp .env.example .env.local
# .env.local already has VITE_API_URL=http://localhost:3000
npm install
npm run dev
```

Open <http://localhost:5173>. The app loads, todos appear, add/delete/toggle all work.

If this works: you're ready to push. If not: **stop**, fix locally, then come back.

Push to GitHub: Monorepo vs Separate Repos

Monorepo (one repo)

```
your-project/  
  backend/  
    src/ ...  
    package.json  
  frontend/  
    src/ ...  
    package.json  
  .gitignore
```

Separate repos (two repos)

```
your-project-backend/  
  src/ ...  
  package.json  
  
your-project-frontend/  
  src/ ...  
  package.json
```

Tonight: monorepo. The trick is telling Render and Vercel which subfolder to deploy (“Root Directory”).

Push Commands

```
git add backend/ frontend/ .gitignore
git status          # confirm .env is NOT staged
git commit -m "Add deploy-ready backend + frontend"
git push
```

Note: Double-check `git status` before committing. If you see `.env` listed, your `.gitignore` is wrong. Fix it before pushing — once a secret is on GitHub, treat it as compromised.

Step 1: Neon (DB) — Where We Are

1. NEON <-- we are here
2. Render
3. Vercel
4. Wire-up

Goal: end with a DATABASE_URL you can paste into Render's env vars.

Time: ~3 minutes.

Neon: Sign Up + Create Project

1. Go to `neon.tech` → **Sign in with GitHub** → authorize
2. **Create Project:**
 - ▶ Name: anything (`your-project-db`)
 - ▶ Postgres version: default
 - ▶ **Region:** pick one close to Render — for NYC use **US East (Ohio)** or **US East (N. Virginia)**

Why region matters: every query travels Render → Neon. Same region $\approx 5\text{ms}$. Different continent $\approx 150\text{ms}$.

Neon: Create Project

 NEON

Dashboard

Projects

Branches

SQL Editor

Integrations

Settings

Billing

Docs

Support

sourya@example.com

Personal

Create a project

Deploy a new Postgres project in seconds.

Project name
Choose a name for your project.

todo-app

Region
Select the region closest to your users.

US East (N. Virginia)

Low latency for users in the United States

Postgres version
Select the Postgres version for this project.

PostgreSQL 16

Recommended

Neon will automatically apply minor version updates.

Cancel

Create Project

Get the Pooled Connection String

On the project dashboard, find the “Connection string” box.

1. Toggle to **Pooled connection** (NOT Direct)
2. Click the eye icon to reveal it
3. Copy it

Why pooled? Render handles many short-lived HTTP requests. Pooled connections handle many concurrent users without exhausting Postgres connection slots.

Neon: Pooled Connection String

NEON

todo-app
Project

Dashboard

Branches

SQL Editor

Tables

Logs

Monitoring

Integrations

Settings

Usage

Billing

Team

Help

Docs

todo-app ACTIVE

Project ID: prj_8x7u3u6l8j3n

Overview

Branches 2

Compute

Storage

Settings

Connection string

Pooled ? ☒

Copy and paste this connection string into your application.

Connection string

Connection details

```
postgresql://neondb_owner:.....@ep-cool-snowflake-12345678-pooler.us-east-1.aws.neon.tech/neondb?sslmode=require
```

Don't commit your connection string.

Use environment variables to store connection strings and other secrets.

Documentation: [Connecting to Neon](#)

[Learn about connection pooling](#)

Project usage

Data transfer (this month)

31 MB

↑ 12% vs last month

Quickstart

Get started with Neon in a few steps.

[View quickstart guide](#)

CSCI 39548

33

Apply Schema to Neon

```
cd backend
echo 'DATABASE_URL="<paste pooled url>"' > .env
echo 'FRONTEND_ORIGIN="http://localhost:5173"' >> .env

npx prisma migrate deploy
```

Expected: Applied migration XXXXX_init (or “No pending migrations”).

✓ **Step 1 done:** Neon project exists, pooled string copied, migrate deploy succeeded.

Step 2: Render (Backend) — Where We Are

1. Neon done (DB URL in hand)
2. RENDER <-- we are here
3. Vercel
4. Wire-up

Goal: end with a live `/api/health` returning `{"status":"ok"}`.

Time: ~10 minutes (most of it watching Render build).

Render: Sign Up + New Web Service

1. Go to `render.com` → **Get Started for Free** → **GitHub** → authorize
2. Dashboard: **New** + (top right) → **Web Service**
3. **Build and deploy from a Git repository** → **Next**
4. Find your repo → **Connect**

If your repo isn't shown, click "Configure account" to grant Render access.

You land on the service config screen. 7 fields + env vars. Get them right BEFORE clicking Create.

Render Config (1 of 3): Name + Language + Branch + Region

Field	Value	Why
Name	your-app-backend	Becomes part of URL
Language	Node	May guess Go — change it
Branch	main	Which branch to deploy
Region	Virginia (US East)	Match Neon region

Render: Service Config

render

+ New

Dashboard

Web Services

Static Sites

Databases

Cron Jobs

Background Workers

Blueprints

Environment Groups

Settings

Billing

Docs

Community

Changelog

Learning Center

Sourya Saha

Dashboard > csci39548-deploy-demo > Settings

Connect

Manual Deploy

csci39548-deploy-demo

Live

Web Service • Free Plan

Events

Logs

Deploys

Metrics

Settings

Environment

Shell

Scaling

Disks

Advanced

Settings

Manage your web service settings.

Name

csci39548-deploy-demo

Unique identifier for the service. Becomes the URL prefix: `https://<name>.onrender.com`.. Lowercase, hyphens, no spaces.

Language

Node

Tells Render which runtime to install (Node vs Go vs Python vs Ruby vs Docker).. Render auto-detects from `package.json` but sometimes guesses wrong — pin to Node explicitly.

Branch

main

Which Git branch Render watches. Pushes to this branch trigger auto-redeploy.

Region

Virginia (US East)

Physical datacenter location. Should match the database region (Neon is in Virginia) to minimize backend ↔ DB latency. Other choices: Oregon, Frankfurt, Singapore, Ohio.

Save Changes

CSCI 39548

38

Render Config (2 of 3): Root Directory

Root Directory: your backend subfolder (e.g. backend or demos/deploy-demo/backend)

- ▶ If blank, Render builds at repo root and finds no `package.json` → instant build failure
- ▶ Setting Root Directory tells Render “treat THIS subfolder as the project”

Note: For monorepos this is the #1 trap. Always set Root Directory.

Render: Root Directory

The screenshot shows the Render dashboard with a sidebar on the left containing navigation links: New, Dashboard, Web Services, Static Sites, Databases, Cron Jobs, Background Workers, Blueprints, Environment Groups, Settings, Billing, Docs, Community, and Changelog. The main content area is titled 'Settings' and includes a breadcrumb trail: Dashboard > csci39548-deploy-demo > Settings. At the top right are 'Connect' and 'Manual Deploy' buttons. The 'Region' is set to 'Virginia (US East)' with a dropdown arrow. Below it, a text box explains: 'Physical datacenter location. Should match the database region (Neon is in Virginia) to minimize backend ↔ DB latency. Other choices: Oregon, Frankfurt, Singapore, Ohio.' The 'Root Directory' is set to 'demos/deploy-demo/backend' in a text box. Below it, a text box explains: 'Tells Render which subfolder of the repo to treat as the project. Render runs `npm install`, `npm run build`, and `npm run start` from inside this folder, not from the repo root.' A blue information box titled 'Why this matters' contains the following text: 'In a monorepo — one Git repo that contains multiple apps in different folders (e.g. `backend/` and `frontend/` side by side) — Render needs to know which folder is “the app.” If Root Directory is left blank, Render builds at repo root, finds no `package.json` there, and the build fails instantly. This is the #1 silent failure point for first-time monorepo deploys. Always set Root Directory before clicking Create.' At the bottom right is a 'Save Changes' button. The bottom of the sidebar shows a 'Learning Center' link and a user profile for 'Sourya Saha'.

render

+ New

Dashboard

Web Services

Static Sites

Databases

Cron Jobs

Background Workers

Blueprints

Environment Groups

Settings

Billing

Docs

Community

Changelog

Learning Center

S Sourya Saha

Dashboard > csci39548-deploy-demo > Settings

Connect Manual Deploy

Region Virginia (US East) Physical datacenter location. Should match the database region (Neon is in Virginia) to minimize backend ↔ DB latency. Other choices: Oregon, Frankfurt, Singapore, Ohio.

Root Directory demos/deploy-demo/backend Tells Render which subfolder of the repo to treat as the project. Render runs `npm install`, `npm run build`, and `npm run start` from inside this folder, not from the repo root.

Relative to the repository root

Why this matters

In a monorepo — one Git repo that contains multiple apps in different folders (e.g. `backend/` and `frontend/` side by side) — Render needs to know which folder is “the app.” If Root Directory is left blank, Render builds at repo root, finds no `package.json` there, and the build fails instantly.

This is the #1 silent failure point for first-time monorepo deploys.

Always set Root Directory before clicking Create.

Save Changes

Render Config (3 of 3): Build + Start Commands

Build Command:

```
npm install && npx prisma generate && npx prisma migrate deploy
```

Start Command:

```
npm run start
```

Instance Type: Free (512 MB RAM, sleeps after 15 min idle)

`npm install` gets deps, `prisma generate` produces the typed client, `prisma migrate deploy` applies pending migrations to Neon. All three in one shot, every deploy.

Render Env Vars + the devDependencies Trap

Scroll to **Environment Variables**. Add **two**:

Key	Value	Why
DATABASE_URL	Neon pooled string	Tells Prisma where the DB is
NPM_CONFIG_PRODUCTION	false	Forces devDeps install

- ▶ Do **NOT** add FRONTEND_ORIGIN yet — no Vercel URL. Step 4.
- ▶ Do **NOT** add PORT — Render assigns it automatically.

Why NPM_CONFIG_PRODUCTION=false? Render's default NODE_ENV=production skips devDeps. Our tsx and prisma CLI live there → build fails without this.

Render: Health Check Path + Create

Scroll to **Advanced** → expand → set:

Health Check Path: `/api/health`

Render polls this every ~30s. Returns 200 → “Live” (green). Non-200 for too long → “Unhealthy” (red).

Now click **Create Web Service**. Render starts cloning + building.

Render: Health Check Path

The screenshot displays the Render dashboard settings for a deployment named 'csci39548-deploy-demo'. The left sidebar contains navigation links: New, Dashboard, Web Services, Static Sites, Databases, Cron Jobs, Background Workers, Blueprints, Environment Groups, Settings, Billing, Docs, Community, and Changelog. The main content area shows the 'Settings' page with the following configurations:

- Root Directory:** demos/deploy-demo/backend. Relative to the repository root. Tells Render which subfolder of the repo to treat as the project. Render runs `npm install`, `npm run build`, and `npm run start` from inside this folder, not from the repo root.
- Build Command:** npm install && npx prisma generate && npx prisma migrate deploy. Command to build your app.
- Start Command:** npm start. Command to start your app.
- Pre-Deploy Command:** (optional). Command to run before each deploy. Useful for migrations, etc.
- Health Checks:** Expanded section showing the **Health Check Path** as `/api/health`. The path Render will request to check if your service is healthy. The URL path Render hits every ~30 seconds to verify your service is alive. Must return HTTP 200 with any body (we return `{ "status": "ok" }`). If it returns anything else for too long, Render marks the service "Unhealthy" and stops sending traffic.

A 'Save Changes' button is located at the bottom of the Health Checks section.

Watch the Build Logs

Render streams the build live. Look for these milestones:

```
==> Cloning from https://github.com/...  
==> Running build command ...  
    added N packages  
    Generated Prisma Client  
    No pending migrations to apply.  
==> Build successful  
==> Running 'npm run start'  
    Prisma demo listening on http://localhost:10000  
==> Your service is live
```

Total time: ~3 minutes for first build. Render gives you a URL. **Copy it.**

Smoke Test Render with curl

```
curl https://csci39548-deploy-demo.onrender.com/api/health
# -> {"status":"ok"}

curl https://csci39548-deploy-demo.onrender.com/api/todos
# -> [{"id":1,"text":"...", "done":false, ...}, ...]
```

If both return JSON, your backend is **live and talking to Neon**.

✓ **Step 2 done:** Render shows “Live,” /api/health returns 200, backend URL copied.

If Step 2 Fails: Common Render Build Errors

Symptom	Cause	Fix
Cannot find <code>tsx</code> or <code>prisma</code>	<code>NPM_CONFIG_PRODUCTION</code> not set	Add env var, redeploy
Cannot find <code>package.json</code>	Root Directory wrong	Set correct subfolder, redeploy
P3009: failed migrations	DB in inconsistent state	Delete migrations/, re-init, push
Service stuck “deploying”	Not listening on <code>process.env.PORT</code>	Fix <code>server.ts</code> , push
Health check failing	Crash or path mismatch	Check logs; confirm <code>/api/health</code>

Step 3: Vercel (Frontend) — Where We Are

```
1. Neon      done
2. Render    done (backend URL in hand)
3. VERCEL    <-- we are here
4. Wire-up
```

Goal: end with a live `.vercel.app` that loads your React app. API calls will still fail — Step 4 fixes that.

Time: ~7 minutes.

Vercel: Sign Up + Plan

1. Go to `vercel.com` → **Sign Up** → **Continue with GitHub** → authorize
2. Pick **Hobby** plan (free, personal projects)
3. Team Name: pick a short slug (shows up in dashboard URLs)

Import the Project

1. **Add New...** → **Project**
2. Find your repo → **Import**

If your repo isn't shown, click “Adjust GitHub App Permissions” to grant Vercel access.

Vercel Project Config

Field	Value	Why
Project Name	your-app	Becomes <code>your-app.vercel.app</code>
Framework Preset	Vite	Auto-detected after Root Dir set
Root Directory	Your frontend subfolder	Same monorepo step as Render
Build / Output	Leave defaults	Vercel knows Vite

When you change Root Directory, Vercel re-scans and Framework Preset usually flips to “Vite” automatically.

Vercel Project Config

 /  Sourya Saha (Hobby) Feedback Changelog Help 

Configure Project

Please provide values for the required Environment Variables and any optional settings.

[Import a different Git Repository →](#)

Project Name
Used to generate the production URL.

 Available https://csc39548-deploy-demo.vercel.app ↗

Framework Preset

 Vite ▾

Root Directory
The directory within your Git repository that contains your project.

Edit

Build Command
The command to run when building your project.



Output Directory
The directory that contains the output of your build.



Install Command
The command to install dependencies.



Environment Variables
Set the environment variables that your project will use.

Add Import

Key	Value	
VITE_API_URL	https://csc39548-deploy-demo.onrender.com	

[Add another ▾](#)

Cancel Deploy

Vercel Env Vars

Expand **Environment Variables**. Add **one**:

Key	Value
VITE_API_URL	Your Render URL. No trailing slash.

Note: Must be VITE_API_URL, not API_URL. Vite strips non-VITE_ vars. If you get it wrong, your frontend fetches from undefined/api/todos.

Deploy + First Smoke Test

Click **Deploy**. Vercel clones → `npm install` → `vite build` → publishes to CDN.
~60 seconds. Confetti.

Open the Vercel URL:

- ▶ ✓ The React app **loads** (layout, fonts, components render)
- ▶ × Todos **fail to load** — DevTools Console shows red CORS errors

That's expected. We haven't told the backend that your Vercel URL is allowed. **Step 4 fixes it.**

✓ **Step 3 done:** Vercel shows "Ready," UI loads, Vercel URL copied.

Step 4: Wire-Up (CORS) — Where We Are

```
1. Neon      done
2. Render    done
3. Vercel     done (frontend URL in hand)
4. WIRE-UP    <-- we are here
```

Goal: tell the backend that your Vercel URL is an allowed origin.

Time: ~2 minutes.

Set FRONTEND_ORIGIN on Render

1. Open **Render dashboard** → your backend service
2. **Environment** (left sidebar) → **Add Environment Variable**
3. Key: FRONTEND_ORIGIN
Value: `https://csci39548-deploy-demo.vercel.app`
No trailing slash. Must include `https://`.
4. **Save Changes**

Render auto-redeploys (~30 seconds).

The Trailing-Slash Trap

Try this on purpose:

Set `FRONTEND_ORIGIN=https://your-app.vercel.app/` (WITH slash). Save. Wait. Refresh in incognito.

You'll see in DevTools Console:

The 'Access-Control-Allow-Origin' header has a value
'https://your-app.vercel.app/' that is not equal to the supplied origin.

The browser's origin has no slash. The server echoes a slash. String compare fails.

Strict.

Strip the slash. Save. Wait. Refresh. Works. **This is gotcha #2 of every deploy you'll ever do.**

Wire-Up Done

Once Render is “Live” with the correct (no-slash) `FRONTEND_ORIGIN`:

- ▶ The Vercel frontend can fetch from the Render backend
- ▶ The Render backend can query Neon
- ▶ The whole chain is connected

You have a live, public, full-stack web app. Time to verify end-to-end.

End-to-End Smoke Test

Open Vercel URL in **incognito**. Open **DevTools** → **Network**.

#	Action	Expected
1	Reload page	GET /api/todos → 200
2	Add a todo	POST → 201, list refreshes
3	Reload	New todo still there (persisted)
4	Click checkbox	PATCH → 200
5	Delete	DELETE → 204, row gone
6	Console tab	No red errors

All 6 pass = deploy is solid.

Cross-Check in Neon

Open **Neon dashboard** → your project → **Tables** → todos.

You should see the rows you just added/deleted from the Vercel app reflected here in real-time.

This is the proof: the data in your browser came from a Postgres database in AWS Virginia, via an Express server on Render. Three services. One round trip. Your laptop is no longer in the loop.

Cross-Check in Neon

NEON

todo-app / main ACTIVE

Feedback Docs

S

PROJECT

Branches

SQL Editor

Tables

Functions

Schema

Roles

Extensions

Integrations

Connection Details

Usage

Monitoring

Settings

Free Plan Upgrade

3.34 GB / 10 GB

Search tables...

Category 3 rows

Todo 5 rows

_prisma_migrations 6 rows

Todo 5 rows 6 columns

Refresh

Filter

Sort

Insert row +

☐	id integer	text text	done boolean	minutes integer	createdAt timestampz	categoryId integer
☐	1	Buy groceries	false	30	2026-06-29 21:42:47.563+00	1
☐	2	Buy bread	true	15	2026-06-29 21:43:12.104+00	1
☐	3	Read 20 pages	false	40	2026-06-29 21:42:47.563+00	NULL
☐	4	Push to GitHub	true	10	2026-06-30 18:34:22.456+00	2
☐	5	Deploy to Vercel	false	25	2026-06-30 18:39:43.789+00	2

Rows per page: 50

Page 1 of 1

What “Deployed” Means Now

- ▶ A public URL anyone can hit ✓
- ▶ A real database storing real data ✓
- ▶ A real backend serving real requests ✓
- ▶ A GitHub-driven workflow: push to `main` → both auto-rebuild ✓

Share the URL. Put it on your resume. Add it to your portfolio.

You shipped a real full-stack web app.

Environment Variables: Master Reference

Where	Variable	Value	Gotcha
Backend .env	DATABASE_URL	Neon pooled string	Pooled, not Direct
Backend .env	FRONTEND_ORIGIN	http://localhost:5173	—
Render	DATABASE_URL	Neon pooled string	Same as local
Render	FRONTEND_ORIGIN	Vercel URL	No trailing slash
Render	NPM_CONFIG_PRODUCTION	false	Required for tsx/prisma
Render	PORT	(DO NOT SET)	Render injects it
Frontend .env.local	VITE_API_URL	http://localhost:3000	Must start with VITE_
Vercel	VITE_API_URL	Render URL	No trailing slash

Gotcha 1 and 2 (The Unforgivable Ones)

Gotcha 1: `NPM_CONFIG_PRODUCTION` not set on Render.

- ▶ Symptom: build fails with “Cannot find module `tsx`” or “`prisma`”
- ▶ Fix: add `NPM_CONFIG_PRODUCTION=false` env var. Redeploy.

Gotcha 2: Trailing slash on `FRONTEND_ORIGIN`.

- ▶ Symptom: CORS error after wire-up. Message shows URL with slash $\neq$ without slash.
- ▶ Fix: strip the slash. Render auto-redeploys.

Gotcha 3 and 4

Gotcha 3: Vercel env var change does NOT auto-redeploy.

- ▶ Symptom: added `VITE_API_URL`, but app still fetches from `localhost:3000`
- ▶ Fix: Vercel dashboard → Deployments → “...” → Redeploy

Gotcha 4: Render cold start = 30s first request.

- ▶ Symptom: first hit after quiet stretch takes 30 seconds
- ▶ Cause: free tier sleeps after 15 min idle
- ▶ Fix: accept it, pay \$7/mo for Starter, or use UptimeRobot

Gotcha 5 + Dev Tips

Gotcha 5: `tsx watch` **does NOT** watch `.env`.

- ▶ Symptom: edit `.env` locally, server doesn't pick up new value
- ▶ Fix: stop server (`Ctrl+C`), `npm run dev` again

Day-to-day patterns:

- ▶ `git push` to main → Render rebuilds (~3 min), Vercel rebuilds (~1 min). Both automatic.
- ▶ Render auto-redeploys on env var change. Vercel doesn't.
- ▶ Schema changes: `prisma migrate dev` locally → `commit prisma/migrations/` → `push` → Render runs `migrate deploy`.

What You Got, Summarized

You change...	What auto-happens
Backend code → <code>git push</code>	Render auto-rebuilds (~3 min)
Frontend code → <code>git push</code>	Vercel auto-rebuilds (~1 min)
Backend env var in Render UI	Render auto-redeploys (~30 sec)
Frontend env var in Vercel UI	Does NOT auto-redeploy — trigger manually
<code>prisma/schema.prisma</code> change	Locally migrate, commit, push → Render runs migrate deploy

You no longer “log into a server.” You push to `main`. Everything else is plumbing.

Final Project Deployment Ask

Every final-project team submits **one** Vercel URL.

- ▶ Vercel URL is what graders will open
- ▶ It must work in an **incognito window** (no logged-in state assumed)
- ▶ README of your final-project repo's first section: paste the Vercel URL

If you can do this for the final project, you've shipped a real full-stack web app.

Going Deeper (After the Course)

- ▶ **Custom domain:** Vercel → Settings → Domains → add `yourname.com`
- ▶ **Always-on backend:** Render Starter plan (\$7/mo) removes cold starts
- ▶ **Preview deployments:** every PR gets its own preview URL on both platforms
- ▶ **Multiple environments:** branch deploys → separate Neon DBs via per-environment env vars
- ▶ **Observability:** Sentry / LogRocket for error tracking
- ▶ **DB backups:** Neon → Branches → restore from any point in last 7 days (free)

Out of scope for tonight, but the path is here when you want it.